

SearchIQ Typeahead System

Single-file submission report for the Search Typeahead System. This report consolidates architecture, dataset loading, APIs, design decisions, trade-offs, performance evidence, and requirement mapping for the final submission.

Runtime	Primary data size	Source of truth	Cache nodes
Node.js 24+	100,000 queries	SQLite	3 logical nodes

1. Executive Summary

SearchIQ is a locally runnable typeahead/autocomplete system. It returns top prefix suggestions, supports popularity-based and recency-aware ranking, uses a cache-before-database read path, and reduces write pressure through batched query-count updates.

The system is intentionally locally reproducible. SQLite stores the durable query-count data, three in-memory Map objects simulate logical cache nodes, a consistent-hash ring assigns prefix keys to cache owners, and a batch writer aggregates repeated searches before flushing them to SQLite.

The design focuses on the core HLD concepts required for the project: low-latency reads, cache distribution, ranking freshness, and write amplification reduction.

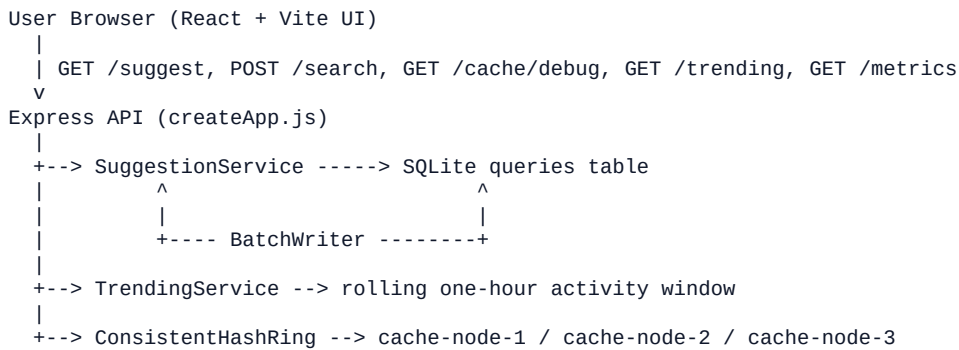
2. Requirement-to-Implementation Mapping

Requirement	Implementation	Evidence
100,000 query,count dataset	Deterministic CSV generator and SQLite bulk ingestion.	src/scripts/generateDataset.js, src/scripts/ingestDataset.js, npm run seed
Search UI and suggestion dropdown	React + Vite UI with debounce, loading/error states, submit button, Enter support, and keyboard-friendly suggestions.	src/App.jsx, src/styles/app.css
GET /suggest?q=prefix	Returns at most 10 case-insensitive prefix matches, sorted by count in basic mode.	src/api/createApp.js, src/services/suggestionService.js
POST /search	Returns { "message": "Searched" }, records recent activity, buffers DB updates, and invalidates affected prefixes.	src/services/searchService.js, src/services/batchWriter.js
GET /cache/debug?prefix=prefix	Shows owning cache node, hash, status, TTL, and node-local stats.	src/api/createApp.js, src/services/distributedCache.js

Design / scoring requirement	Implementation	Evidence
Cache-before-database	Cache-aside flow: lookup cache first, fallback to SQLite on miss/expiry, then store response with TTL.	suggestionService.js, distributedCache.js
Distributed cache using consistent hashing	Three logical cache nodes with virtual nodes and deterministic key ownership for prefixes.	consistentHashRing.js, distributedCache.js
Trending searches	Same /suggest API supports trending mode using score = allTimeCount + recentCountLastHour * 50.	trendingService.js, suggestionService.js
Batch writes	In-memory aggregation buffer with periodic/size-based flush and SQLite transaction writes.	batchWriter.js, database.js
Performance evidence	Seed timings, benchmark helper, cache hit rate, and write-reduction metrics.	docs/performance-report.md, src/scripts/benchmarkSuggest.js

3. System Architecture

The application is divided into frontend, API, storage, cache, ranking, and write-buffering concerns. This keeps the implementation modular while making the data path easy to inspect.



End-to-End Suggestion Flow

- The user types a prefix and the React UI waits for a 280ms debounce.
- The frontend calls GET /suggest?q=prefix&ranking=mode.
- The backend normalizes the prefix and builds a cache key such as basic:iph or trending:iph.
- The consistent-hash ring chooses the logical cache node that owns that key.
- On cache hit, the API returns cached suggestions without touching SQLite.
- On cache miss or expiry, the API queries SQLite, merges pending increments, ranks results, caches the response, and returns it.

4. Dataset Source and Loading

The dataset is generated locally and deterministically; no external download is required. The generated CSV follows the required query,count format and contains exactly 100,000 records for the full seed.

Command	Purpose
npm run seed:small	Generate and ingest 5,000 rows for quick development resets.
npm run seed	Generate and ingest the full 100,000-row dataset used for submission checks.

- CSV generation logs start, progress, and completion.
- SQLite schema initialization is logged separately.
- Ingestion uses batched SQLite transaction writes.
- Generated CSV and database files are local artifacts and are excluded from the submission ZIP.

5. API Documentation

API	Purpose	Key behavior
GET /suggest?q=prefix	Fetch suggestions.	Returns up to 10 prefix matches using basic count-desc ranking by default.
GET /suggest?q=prefix&ranking=trending	Fetch recency-aware suggestions.	Uses score = allTimeCount + recentCountLastHour * 50.
POST /search	Submit a search.	Returns { "message": "Searched" }, records recent activity, buffers the update, and invalidates affected cache keys.
GET /cache/debug?prefix=prefix	Debug cache routing.	Shows assigned logical node, hit/miss/expired status, TTL, and node stats.
GET /trending?limit=n	Read live trends.	Returns rolling-window trend data for the UI signals panel.
GET /metrics	Read operational metrics.	Reports cache hit rate, DB reads/writes, batch buffer state, writes avoided, and indexed query count.

6. Design Choices and Trade-offs

Decision	Why chosen	Trade-off	Production alternative
SQLite source of truth	Simple, durable, local, and easy to validate for 100,000 rows.	Not horizontally scalable.	PostgreSQL, DynamoDB, Cassandra, or another scalable primary store.
SQL prefix lookup	Readable and sufficient at this dataset size when paired with caching.	Less efficient than a dedicated prefix index at very large scale.	Trie, prefix index, OpenSearch/Elasticsearch, or precomputed top-K tables.
Map-based logical cache nodes	Demonstrates cache ownership, TTL, invalidation, and consistent hashing without Docker/Redis setup.	Cache is process-local and lost on restart.	Redis Cluster or Memcached.
Consistent-hash ring	Stable key-to-node routing and reduced remapping compared with modulo hashing.	Still a simulation in this local implementation.	Redis Cluster slots or a production consistent-hash client.
One-hour trending window	Keeps recent activity temporary and explainable.	Fixed boost is not personalized or ML-based.	Exponential decay, hourly buckets, personalization, click-through signals.
In-memory batch buffer	Aggregates repeated queries and reduces write amplification.	Buffered increments may be lost before flush.	Kafka, Redis Streams, SQS, or a database-backed queue.

7. Performance Report

Measurements were collected on the local Windows environment used for final validation. These are measured values from the current implementation, not placeholders.

Metric	Measured value
npm run seed:small	5,000 rows; CSV generation 55ms; ingestion 870ms; total 975ms.
npm run seed	100,000 rows; CSV generation 546ms; ingestion 4323ms; total 4879ms.
Cold /suggest?q=iph	10.94ms wall-clock; 6.25ms API-reported latency.
Warm /suggest?q=iph p95 over 25 requests	3.28ms wall-clock; 0.22ms API-reported latency.
Warm /suggest?q=iph ranking=trending p95 over 25 requests	2.05ms wall-clock; 0.18ms API-reported latency.
Cache hit rate after warm-up	96%.
Batch evidence screenshot	16 submissions, 6 flushes, 10 writes avoided, 63% write reduction.

Interpretation

Repeated autocomplete prefixes are a strong fit for prefix-level caching. The warm request latency is significantly lower because cache hits avoid repeated SQLite reads. The batch writer reduces write pressure by aggregating repeated searches before flushing them to the database.

8. Screenshot Evidence

The screenshots are kept in the repository under docs/screenshots/. They are not embedded as large pages in this report so the PDF stays readable and focused, while the repository still provides the visual evidence required for review.

Screenshot file	What it proves
docs/screenshots/home.png	Initial SearchIQ screen with search interface, indexed query count, cache node count, and core evidence panels.
docs/screenshots/suggestions.png	Prefix suggestions for iph, showing top matching queries and counts.
docs/screenshots/cache-hit.png	Repeated prefix lookup showing source = cache, cache status = hit, owning cache node, latency, and TTL.
docs/screenshots/trending.png	Trending mode showing recency-aware ordering and score values.
docs/screenshots/batch-metrics.png	Batch-write metrics showing submissions, flushes, pending entries, writes avoided, and write reduction.

9. Validation Summary

Validation item	Result
npm test	Passed 13/13 tests.
npm run build	Passed.
npm run seed:small	Passed; inserted 5,000 rows.
npm run seed	Passed; inserted 100,000 rows.
npm run benchmark	Collected cold/warm /suggest latency and cache hit-rate evidence.

10. Conclusion

This project satisfies the required search typeahead scope: a 100,000-row dataset, a working UI, prefix suggestions, query submissions, cache-before-database reads, consistent-hash cache distribution, trending ranking, batched writes, metrics, tests, screenshots, and a consolidated project report. The implementation remains locally reproducible while clearly demonstrating the key HLD concepts.